ACTIVIDAD DE COMPROBACIÓN N.° 3 · EVALUACIÓN POR PRODUCTO

Sistema con almacenamiento de datos

Parte 1: corrección de código · Parte 2: préstamo de libros de una biblioteca escolar

Módulo	Diseño de algoritmos · CSTI12010
Nombre de la persona estudiante	_____
Identificación	_____
Oportunidad de evaluación	1.ª (X) 2.ª ( )
Fecha · tiempo	Jueves 5 de noviembre de 2026 · 3 horas
Hora de inicio · finalización	___1:30pm_____ · ____4:30pm____
Persona docente	Giovanni Antonio Coto Calderón

Descripción general

El proyecto consiste en la solución de un caso asignado por la persona docente, que incorpore elaborar un sistema que realiza creación, modificación, actualización y borrado de registros, con el paradigma de programación estructurada, según los requerimientos establecidos. El proyecto se elabora de forma individual.

En este caso los registros se guardan en **arreglos** (en memoria); los indicadores de manejo de archivos se marcan **NA (no aplica)**.

Instrucciones específicas

a) Lea cuidadosamente las instrucciones dadas por la persona docente y los indicadores de esta evaluación.

b) Realice el trabajo de forma ordenada, utilizando buenas prácticas de desarrollo de software.

c) Emplee el paradigma de programación estructurada en la elaboración de la solución desarrollada.

d) Demuestre ética en la creación de la solución, no se aceptará plagio de la web u otras personas estudiantes.

e) Utilice el siguiente nombre para el zip de la entrega: Proyecto03_PrimerNombre_PrimerApellido.zip. Es muy importante que la carpeta lleve su primer nombre y apellido para poder identificarlo correctamente.

f) Trabaje de forma individual durante las 3 horas. Compile con gcc -std=c11 -Wall -Wextra.

Material y recursos permitidos

- **Sí se permite:** solo su material escrito a puño y letra (sus apuntes en papel).
- **Internet: no se permite**, salvo al inicio para descargar estas instrucciones y al final para subir su trabajo a GitHub y enviar el correo electrónico con la carpeta comprimida.
- **No se permiten celulares ni ninguna otra fuente:** material impreso, archivos o programas guardados en la computadora o en memorias USB, páginas web, inteligencia artificial ni ayuda de otras personas.

Estructura de la actividad (3 horas)

- **Parte 1 · Corrección de código (sugerido: 40 minutos).** Dos programas cortos con 2 errores cada uno: encuéntrelos, corríjalos y explíquelos en comentarios.
- **Parte 2 · Ejercicio (sugerido: 2 horas y 20 minutos).** Programar el sistema de préstamo de libros de una biblioteca escolar.

Parte 1 · Corrección de código

La persona docente le entrega los archivos **correccion1_puntos.c** y **correccion2_buscar.c**. Cada uno tiene **2 errores**. Para cada archivo:

- Compílelo con gcc -std=c11 -Wall -Wextra, ejecútelo con los datos de prueba y compare con el resultado esperado.
- Encuentre y corrija los 2 errores. Lea las advertencias del compilador, pero recuerde que hay errores que el compilador no avisa.
- Arriba de cada línea corregida escriba un comentario con **qué pasaba**, **por qué** y **cuál es la corrección**, por ejemplo: // ERROR: ... // POR QUE: ... // CORRECCION: ...
- Compruebe que el programa corregido compila sin advertencias y da el resultado esperado.

Corrección 1 · Puntos de cliente frecuente (correccion1_puntos.c)

Un comercio da 1 punto por cada 1000 colones de compra. El programa lee 3 compras, suma los puntos de cada una con la función sumarPuntos y muestra el total.

correccion1_puntos.c

```c
#include <stdio.h>

void sumarPuntos(int puntos, int compra) {
    puntos = puntos + compra / 1000;
}

int main(void) {
    int puntos = 0, compra, i;
    for (i = 1; i <= 3; i++) {
        printf("Compra %d: ", i);
        scanf("%d", compra);
        sumarPuntos(puntos, compra);
    }
    printf("Puntos acumulados: %d\n", puntos);
    return 0;
}
```

Datos de prueba	Compras 5000, 12500 y 800
Resultado esperado	Puntos acumulados: 17 (5 + 12 + 0)

Pista: un error lo avisa el compilador (lea el mensaje de la línea del scanf). El otro no: el total sale siempre 0; revise si sumarPuntos recibe el total por valor o por dirección.

Corrección 2 · Buscar un producto (correccion2_buscar.c)

Una pulpería guarda 4 productos con su precio. El programa lee el nombre de un producto, lo busca con la función buscar y muestra su precio o «No existe».

correccion2_buscar.c

```c
#include <stdio.h>
#include <string.h>
#define N 4

int buscar(const char nombres[][15], int n, const char buscado[]) {
    int i;
    for (i = 0; i <= n; i++)
        if (nombres[i] == buscado)
            return i;
    return -1;
}

int main(void) {
    char nombres[N][15] = {"Arroz", "Frijoles", "Cafe", "Azucar"};
    double precios[N] = {1200, 1500, 3200, 900};
    char buscado[15];
    int pos;
    printf("Producto: ");
    scanf("%14s", buscado);
    pos = buscar(nombres, N, buscado);
    if (pos == -1)
        printf("No existe\n");
    else
        printf("%s cuesta %.2f\n", nombres[pos], precios[pos]);
    return 0;
}
```

Datos de prueba	Cafe y luego Pan (dos ejecuciones)
Resultado esperado	Cafe cuesta 3200.00 · No existe

Pista: ninguno de los dos lo avisa el compilador. Uno está en cómo se comparan dos cadenas; el otro, en hasta dónde llega el for.

Parte 2 · Caso: préstamo de libros de una biblioteca escolar

La biblioteca de un colegio necesita un sistema para controlar el préstamo de sus libros. Puede registrar **hasta 5 libros**. De cada libro se guarda: el **código** (100 a 999, no se repite), el **título** (una sola palabra, hasta 19 letras), los **ejemplares disponibles** y si el libro está **activo** o dado de baja.

El programa inicia con **3 libros ya cargados** (tabla de abajo). Debe mostrar un **menú** que se repite hasta elegir 0; toda opción o dato fuera de rango se vuelve a pedir.

Opción del menú	Qué debe hacer
1 Incluir libro	Si hay espacio, pide código, título y ejemplares (1 a 20). Si el código ya existe en un libro activo, avisa y no lo incluye. Un libro nuevo inicia activo y con todos sus ejemplares disponibles.
2 Consultar libros	Muestra en columnas el código, el título y los disponibles de cada libro activo, y al final cuántos libros activos hay.
3 Prestar un ejemplar	Pide el código. Si hay ejemplares disponibles, resta uno y muestra cuántos quedan; si no hay, avisa.
4 Dar de baja un libro	Pide el código y marca el libro como dado de baja: ya no aparece en consultas ni búsquedas.
5 Buscar libro	Pide el código y muestra el código, el título y los disponibles del libro.
0 Salir	Muestra un mensaje de despedida y termina.

En las opciones 3, 4 y 5, si el código no corresponde a un libro activo, el programa avisa «No existe un libro activo con el código …» y vuelve al menú.

Libros cargados al iniciar:

Código	Título	Disponibles
101	Quijote	3
102	Principito	2
103	Cocori	4

Plantilla de inicio: biblioteca_plantilla.c

La persona docente le entrega **biblioteca_plantilla.c**. Ya trae las bibliotecas, las constantes, los arreglos con los 3 libros, los **prototipos**, **leerEntero** y el **menú completo** (main), con las llamadas a las funciones.

Usted programa el **cuerpo de las 5 funciones** marcadas como TAREA 1 a TAREA 5, en ese orden, siguiendo los pasos escritos en cada una: buscarCodigo, incluir, consultar, prestar y darDeBaja.

Compile después de cada función. Al inicio aparecen advertencias «unused parameter»: desaparecen al completar las funciones.

Complete también el encabezado de comentarios y agregue sus comentarios (qué hace, cómo y para qué).

Tarjeta de referencia · funciones y punteros: para que una función cambie un dato de main, la llamada envía la **dirección** (&) y la función la recibe como **puntero** (*).

Llamada (en main)	Función (prototipo)	Dentro de la función
`incluir(..., &n, ...);`	`void incluir(..., int *n, ...)`	`(*n)++;  con paréntesis`
`prestar(&disp[pos])`	`int prestar(int *disponibles)`	`if (*disponibles == 0) ... (*disponibles)--;`
`darDeBaja(&activo[pos]);`	`void darDeBaja(int *activo)`	`*activo = 0;`
`consultar(cod, tit, disp, activo, n);`	`void consultar(const int cod[], ...)`	`cod[i], tit[i] (los arreglos van sin &)`

Cadenas: strcpy(destino, origen) para copiar y strcmp(a, b) == 0 para comparar (nunca = ni ==).

Requisitos técnicos

- Constantes (#define) para los límites del enunciado.
- **Arreglos paralelos** para código, título, disponibles y estado (1 activo, 0 dado de baja), y una variable con la cantidad de posiciones usadas.
- Menú con **do-while** y **switch**; validación de todos los datos numéricos.
- Programa **modular** con prototipos antes de main y, como mínimo, las funciones de la tabla siguiente. Los nombres pueden variar, pero cada tarea debe estar en su función y las marcadas deben recibir la **dirección** del dato que cambian.
- Encabezado de comentarios (enunciado, qué se logra, cómo se logra, resultado esperado) y comentarios que expliquen qué hace cada parte, cómo y para qué.

Función	Qué hace	Recibe por dirección
`leerEntero`	Pide un entero entre un mínimo y un máximo y lo vuelve a pedir si está fuera de rango (do-while).	—
`buscarCodigo`	Búsqueda secuencial de un libro ACTIVO por código; devuelve la posición o -1.	—
`incluir`	Guarda el libro en la primera posición libre y aumenta la cantidad.	La cantidad de libros (int *n)
`consultar`	Muestra la tabla de libros activos y cuántos son.	—
`prestar`	Resta un ejemplar disponible; devuelve 1 si pudo o 0 si no había.	Los disponibles del libro
`darDeBaja`	Pone el estado del libro en 0.	El estado del libro

Caso de prueba

Antes de entregar, pruebe su programa con esta secuencia, en este orden. Los resultados de la derecha son los que debe obtener.

Paso	Acción	Resultado esperado
1	2 Consultar	Los 3 libros iniciales; libros activos: 3.
2	1 Incluir, código 102	El código 102 ya existe.
3	1 Incluir: 104, Platero, 25 (inválido) y luego 2	Pide de nuevo los ejemplares; libro incluido (4 de 5).
4	3 Prestar 102, tres veces	Disponibles 1, luego 0; la tercera vez: no hay ejemplares disponibles.
5	4 Dar de baja 101	Libro Quijote dado de baja.
6	5 Buscar 101	No existe un libro activo con el código 101.
7	5 Buscar 103	103 Cocori: 4 disponibles.
8	Opción 9 (inválida) y luego 2 Consultar	Pide de nuevo la opción; muestra 102 (0), 103 (4) y 104 (2); libros activos: 3.

Entrega

- Parte 1: **correccion1_puntos.c** y **correccion2_buscar.c** corregidos y comentados.
- Parte 2: programa fuente **biblioteca.c** y ejecutable **biblioteca.exe** (gcc -std=c11 -Wall -Wextra biblioteca.c -o biblioteca.exe).
- Los cuatro archivos dentro de **Proyecto03_PrimerNombre_PrimerApellido.zip**.
- Al final: suba el trabajo a su repositorio de GitHub y envíe por correo electrónico la carpeta comprimida a la persona docente.

Se evalúa con el instrumento de la AC3

Por desempeño: indicadores 1 a 4. Por producto: indicadores 1 a 10, 16 y 20; los indicadores 11 a 15 y 17 a 19 (archivos) se marcan NA. La parte 1 aporta evidencia a los indicadores 1 (compila), 3 (comentarios), 4 y 5 (arreglos) y 10 (punteros); la parte 2, a todos. El detalle está en el documento «Temas y criterios de evaluación».